NETWORK TRAFFIC ANALYSIS USING WIRESHARK

Practical Network Protocol Analysis Report

Submitted by	Vaibhavi Singh
Date	12 October 2026
Tools Used	Wireshark, Windows OS

This report documents a Wireshark packet-capture exercise covering DNS, TCP, TLS, HTTP, and retransmission indicators. The public version intentionally omits detailed packet screenshots and IP-address information for privacy.

Public report version — detailed packet screenshots and sensitive packet data are not included.

1. INTRODUCTION

The objective of this project was to capture and analyze live network traffic on a Wi-Fi network using Wireshark. The analysis focused on understanding how data travels between a client device and external servers, including DNS resolution, TCP handshakes, and identifying network performance indicators such as packet retransmissions.

This project demonstrates practical skills in network protocol analysis, packet filtering, and troubleshooting using industry-standard tools.

2. METHODOLOGY

- Started a live packet capture on the Wi-Fi interface using Wireshark.
- Applied display filters to isolate specific protocols (DNS, TCP, TLS, HTTP).
- Analyzed packet details including sequence numbers, flags, and retransmission indicators.
- Documented observations while keeping detailed packet screenshots out of this public version.

3. FILTERS USED

Filter	Purpose
`dns`	View DNS query and response traffic.
`dns.flags.response == 1`	View DNS responses only.
`tcp`	View TCP traffic.
`tcp.flags.syn == 1`	View packets with SYN set.
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	View SYN packets without ACK set.
`tcp.flags.syn == 1 && tcp.flags.ack == 1`	View SYN-ACK packets.
`tcp.flags.ack == 1 && tcp.flags.syn == 0`	View ACK-set, SYN-unset packets; this can include more than the final handshake ACK.
`tcp.analysis.retransmission`	View packets Wireshark flags as retransmissions.
`tls`	View TLS traffic, if present.
`http`	View HTTP traffic, if present.

4. ANALYSIS

4.1 Initial Live Capture

Filter used: `None (raw capture)`

Initial live capture on the Wi-Fi interface showing a mix of TCP, TLS, and ARP traffic. This is the baseline network activity.

4.2 DNS Query (Client to Server)

Filter used: `dns`

The DNS query sent by the client to resolve the domain `clients4.google.com`. The computer is asking the DNS resolver for information about this domain.

4.3 DNS Query Details

Filter used: `dns (packet 244 expanded)`

Detailed view of the DNS query packet. The query type is AAAA (IPv6 address request) for `clients4.google.com`. The transaction ID and recursion-desired flag are also visible.

4.4 DNS Response (Server to Client)

Filter used: `dns.flags.response == 1`

The DNS response shows a CNAME record indicating that `clients4.google.com` is an alias for `clients.l.google.com`, followed by IPv6 address records as described in the supplied notes. Confirm exact answer fields against the response capture.

4.5 TCP Traffic Overview

Filter used: `tcp`

TCP traffic between the device and external servers, including port 443 commonly associated with HTTPS. ACK packets acknowledge TCP sequence-space; they do not alone prove that no data was lost.

4.6 TCP Packet Details (ACK)

Filter used: `tcp (packet 282 expanded)`

Detailed view of a TCP packet. Relative Sequence Number and Acknowledgment Number values may display as 1 depending on Wireshark settings. Interpret these fields in the context of the connection.

4.7 TCP SYN and Retransmission

Filter used: `tcp.flags.syn == 1`

The supplied notes identify packet 7598 as a TCP retransmission. Verify packet details and connection context before concluding it was a retransmitted SYN. A retransmission may be consistent with a delayed or missing response but does not prove the cause.

4.8 TCP Handshake — SYN (Step 1)

Filter used: `tcp.flags.syn == 1 && tcp.flags.ack == 0`

The first step of a TCP connection: a SYN packet. The notes identify a connection attempt to a server on port 80.

4.9 TCP Handshake — SYN-ACK (Step 2)

Filter used: `tcp.flags.syn == 1 && tcp.flags.ack == 1`

The server's SYN-ACK is the second step of the TCP three-way handshake. Confirm it belongs to the same connection as the initiating SYN.

4.10 TCP Handshake Complete & Data Flow (Step 3)

Filter used: `tcp.flags.ack == 1 && tcp.flags.syn == 0`

This filter can include the final ACK and later TCP packets. The notes mention a TLS Client Hello to `settings-win.data.microsoft.com`; verify the connection and packet sequence before concluding the handshake is complete.

4.11 Network Performance Analysis (Retransmissions)

Filter used: `tcp.analysis.retransmission`

Multiple packets were reportedly flagged as retransmissions. These can be associated with packet loss, congestion, delayed acknowledgments, or other conditions. The filter alone does not prove network instability, packet loss, or high latency conclusively.

4.12 TLS Encrypted Data

Filter used: `tls`

TLS traffic was observed. Some packet metadata may be visible, while application content is normally encrypted. TLS traffic is consistent with protected communication, but the filter alone does not establish that all website traffic used HTTPS.

4.13 HTTP GET Request

Filter used: `http`

The notes describe an HTTP GET request for a file ending in `.crl`. Certificate Revocation List files can be used for certificate-revocation checking. Confirm the request in the capture and do not publish sensitive headers or payload data.

5. KEY FINDINGS

- **DNS Resolution:** A query for `clients4.google.com` was observed; the supplied notes describe CNAME and IPv6 address records in the associated response.
- **TCP Handshake:** SYN, SYN-ACK, and ACK filters were used to examine TCP connection establishment. Confirm all three packets belong to the same connection.
- **Retransmissions:** Wireshark's retransmission filter was used to identify packets for further investigation. Retransmissions alone do not prove a specific network fault.

- **TLS:** TLS traffic was observed; encrypted application data is not normally readable from packet contents alone.
- **HTTP:** The notes describe an HTTP GET request for a CRL-related file, which should be confirmed against the capture.

6. CONCLUSION

This project provided practical experience in capturing, filtering, and analyzing network traffic using Wireshark. The investigation covered DNS resolution, TCP three-way handshake concepts, retransmission indicators, TLS traffic, and HTTP traffic. It strengthened understanding of network protocols, packet inspection, and the need to support troubleshooting conclusions with packet-level evidence.

7. PRIVACY NOTE

For privacy and security reasons, detailed screenshots containing packet data and IP addresses have been omitted from this public version. The complete report with screenshots is available upon request for verification purposes. Before sharing evidence, review it for private IPv4/IPv6 addresses, MAC addresses, router details, private domain names, credentials, cookies, and tokens.

END OF REPORT